

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Repaso

Repaso Conceptual

Ejercicios Guiados de Repaso

Soluciones de Referencia

Consolidación y Retroalimentación

Conclusiones

Introducción y Repaso

- **Semana 14, Sesión 1** se centró en:
 - Ajuste lineal con `np.polyfit`
 - Métricas de calidad: residuos, R^2 , χ^2 reducido
 - Ponderación por incertidumbres
 - Introducción a `scipy.optimize.curve_fit`
- **Semana 14, Sesión 2** consolidó:
 - Aplicación práctica de ajustes con incertidumbres
 - Análisis completo con matriz de covarianza
 - Interpolación y selección de modelos
- **Conexión** con la sesión actual: reforzar estos conceptos con ejercicios adicionales

- **Consolidar** el uso de `np.polyfit` y `curve_fit`
- **Reforzar** la interpretación de métricas de calidad del ajuste
- **Practicar** el cálculo y análisis de residuos, R^2 y χ^2_{red}
- **Aplicar** ponderación correcta en ajustes con incertidumbres variables
- **Desarrollar** habilidades de diagnóstico de modelos

Repaso Conceptual

Sintaxis básica

```
coefs = np.polyfit(x, y, deg=1)
```

Retorna coeficientes $[m, b]$ para $y = mx + b$

- **Ventajas:**

- Simple y directo para polinomios
- Soporta ponderación con parámetro w

- **Limitaciones:**

- Solo modelos polinomiales
- No retorna matriz de covarianza directamente

1. **Residuos** ($r_i = y_i - \hat{y}_i$):
 - Diferencia entre datos y modelo
 - Deben distribuirse aleatoriamente alrededor de cero
2. **Coefficiente de determinación** ($R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$):
 - Fracción de varianza explicada por el modelo
 - $R^2 \approx 1$: excelente ajuste
3. **Chi cuadrado reducido** ($\chi^2_{\text{red}} = \frac{1}{\nu} \sum \frac{r_i^2}{\sigma_i^2}$):
 - Evalúa consistencia con incertidumbres
 - $\chi^2_{\text{red}} \approx 1$: ajuste óptimo

Repaso Conceptual $\ni$ Recordatorio: `curve_fit` y Matriz de Covarianza

Sintaxis

```
popt, pcov = curve_fit(func, x, y, sigma=...,  
absolute_sigma=True)
```

- **Ventajas sobre `polyfit`:**
 - Funciona con cualquier función (no solo polinomios)
 - Retorna matriz de covarianza `pcov`
 - Permite extraer incertidumbres: $\sigma_i = \sqrt{\text{pcov}[i, i]}$
- **Parámetros importantes:**
 - `sigma`: incertidumbres en y
 - `absolute_sigma=True`: usar valores absolutos
 - `p0`: valores iniciales para parámetros

Ejercicios Guiados de Repaso

Ejercicios Guiados de Repaso $\ni$ Ejercicio 1:

Péndulo Simple - Periodo vs Longitud



Enunciado

- Un péndulo simple tiene periodo $T = 2\pi\sqrt{\frac{L}{g}}$.
- Para $g = 9.8 \text{ m/s}^2$, se miden periodos para diferentes longitudes:
 - $L = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0] \text{ m}$
 - Generar T con ruido: `sigma` = 0.05 s
- Ajustar el modelo $T = a\sqrt{L}$ usando `curve_fit`.
- Calcular R^2 y χ^2_{red} .
- Comparar a estimado con el valor teórico $2\pi/\sqrt{g} \approx 2.007$.

Conceptos: Modelo no lineal, ajuste con `curve_fit`, validación con teoría física.

Física relevante: Ley del péndulo simple, relación $T \propto \sqrt{L}$.

Ejercicios Guiados de Repaso $\ni$ Ejercicio 2:

Ley de Enfriamiento de Newton



Enunciado

- La temperatura de un objeto enfriándose sigue:
$$T(t) = T_{\text{amb}} + (T_0 - T_{\text{amb}})e^{-kt}$$
- Datos simulados:
 - $t = [0, 5, 10, 15, 20, 25, 30]$ minutos
 - $T_{\text{amb}} = 20\text{C}$, $T_0 = 80\text{C}$, $k = 0.08 \text{ min}^{-1}$
 - Ruido: $\sigma = 1.5\text{C}$
- Ajustar con **curve_fit** para estimar T_{amb} , T_0 y k .
- Graficar datos con barras de error y curva ajustada.
- Reportar parámetros con incertidumbres desde **pcov**.

Conceptos: Ajuste exponencial, múltiples parámetros, extracción de incertidumbres.

Física relevante: Transferencia de calor por convección.

Ejercicios Guiados de Repaso $\ni$ Ejercicio 3:

Resistividad Eléctrica vs Temperatura



Enunciado

- La resistividad de un conductor metálico: $\rho(T) = \rho_0[1 + \alpha(T - T_0)]$
- Datos experimentales:
 - $T = [0, 20, 40, 60, 80, 100] \text{ }^\circ\text{C}$
 - $\rho_0 = 1.68 \times 10^{-8} \Omega \cdot \text{m}$ (cobre a 0°C)
 - $\alpha = 0.00393 \text{ C}^{-1}$, $T_0 = 0\text{C}$
 - Incertidumbres variables: $\sigma_\rho = 0.02 + 0.0005 \cdot T$ ($10^{-8} \Omega \cdot \text{m}$)
- Realizar dos ajustes: SIN y CON ponderación.
- Comparar α estimado en ambos casos.
- Calcular χ^2_{red} para el ajuste ponderado.

Conceptos: Ajuste lineal, ponderación por incertidumbres, comparación de métodos.

Física relevante: Coeficiente térmico de resistividad.

Ejercicios Guiados de Repaso $\ni$ Ejercicio 4:

Análisis Completo de Residuos



Enunciado

- Usar datos del Ejercicio 1 (péndulo simple).
- Calcular y graficar residuos vs L .
- Crear histograma de residuos normalizados: r_i/σ .
- Verificar que:
 - Media de residuos ≈ 0
 - Desviación estándar $\approx \sigma$ esperado
 - Distribución aproximadamente gaussiana
- Identificar si hay patrones sistemáticos en residuos.

Conceptos: Diagnóstico de ajuste, análisis estadístico de residuos, detección de sesgos.

Objetivo: Desarrollar ojo crítico para evaluar calidad de ajustes.

Soluciones de Referencia

Soluciones de Referencia $\ni$ Solución 1:

Péndulo Simple (Parte 1: Ajuste)



```
1 import numpy as np
2 from scipy.optimize import curve_fit
3
4 # Generar datos
5 g = 9.8 # m/s^2
6 L = np.array([0.5, 1.0, 1.5,
7              2.0, 2.5, 3.0])
8 a_true = 2*np.pi/np.sqrt(g)
9 T_true = a_true * np.sqrt(L)
10 sigma = 0.05 # s
11
12 np.random.seed(42)
13 T_obs = T_true + np.random.normal(
14     0, sigma, size=L.size)
15
16 # Definir modelo
17 def pendulo_model(L, a):
18     return a * np.sqrt(L)
```

```
17 # Ajustar
18 popt, pcov = curve_fit(
19     pendulo_model, L, T_obs,
20     sigma=np.full(L.size, sigma),
21     absolute_sigma=True)
22 a_est = popt[0]
23 sigma_a = np.sqrt(pcov[0, 0])
24
25 # Reportar
26 print(f"a teórico = {a_true:.4f}")
27 print(f"a estimado = {a_est:.4f} "
28       f"± {sigma_a:.4f}")
29 print(f"Diferencia = "
30       f"{abs(a_est-a_true):.4f}")
```


Soluciones de Referencia $\ni$ Solución 1:

Péndulo Simple (Parte 2: Métricas)



```
1 # Calcular predicciones
2 T_fit = pendulo_model(L, a_est)
3
4 # Calcular R^2
5 residuos = T_obs - T_fit
6 SS_res = np.sum(residuos**2)
7 SS_tot = np.sum((T_obs - np.mean(T_obs))**2)
8 R2 = 1 - SS_res/SS_tot
9
10 # Calcular chi^2 reducido
11 chi2 = np.sum((residuos/sigma)**2)
12 nu = L.size - 1 # 1 parámetro ajustado
13 chi2_red = chi2/nu
14
15 print(f"\nMétricas de calidad:")
16 print(f"R^2 = {R2:.4f}")
17 print(f" $\chi^2_{red}$  = {chi2_red:.2f}")
18
19 if 0.5 < chi2_red < 2.0:
20     print("✅ Ajuste consistente con incertidumbres")
```

Discusión: $R^2 \approx 1$ indica excelente ajuste. $\chi^2_{red} \approx 1$ confirma que el ruido es consistente con $\sigma = 0.05$ s.

Soluciones de Referencia $\ni$ Solución 2:

Enfriamiento de Newton (Parte 1)



```
1 import numpy as np
2 from scipy.optimize import curve_fit
3
4 # Datos sintéticos
5 t = np.array([0, 5, 10, 15,
6              20, 25, 30])
7 T_amb_true = 20.0
8 T0_true = 80.0
9 k_true = 0.08
10 sigma_T = 1.5
11
12 T_true = (T_amb_true +
13           (T0_true - T_amb_true) *
14           np.exp(-k_true * t))
15 np.random.seed(10)
16 T_obs = T_true + np.random.normal(
17     0, sigma_T, size=t.size)
```

```
16 # Modelo de enfriamiento
17 def enfriamiento(t, T_amb,
18                 T0, k):
19     return (T_amb + (T0 - T_amb) *
20            np.exp(-k * t))
21
22 # Ajustar con valores iniciales
23 p0 = [25, 75, 0.1]
24 popt, pcov = curve_fit(
25     enfriamiento, t, T_obs,
26     p0=p0,
27     sigma=np.full(t.size,
28                  sigma_T),
29     absolute_sigma=True)
30 T_amb, T0, k = popt
```

Soluciones de Referencia $\ni$ Solución 2:

Enfriamiento de Newton (Parte 2)



```
1 # Extraer incertidumbres
2 perr = np.sqrt(np.diag(pcov))
3 sigma_Tamb, sigma_T0, sigma_k = perr
4
5 print("Parámetros ajustados:")
6 print(f"T_amb = {T_amb:.2f} ± "
7       f"{sigma_Tamb:.2f} °C")
8 print(f"T0 = {T0:.2f} ± "
9       f"{sigma_T0:.2f} °C")
10 print(f"k = {k:.4f} ± "
11       f"{sigma_k:.4f} min⁻¹")
12
13 print("\nValores reales:")
14 print(f"T_amb = {T_amb_true} °C")
15 print(f"T0 = {T0_true} °C")
16 print(f"k = {k_true} min⁻¹")
```

```
16 # Graficar
17 import matplotlib.pyplot as plt
18
19 t_fine = np.linspace(0, 30, 100)
20 T_fit_fine = enfriamiento(
21     t_fine, T_amb, T0, k)
22
23 plt.errorbar(t, T_obs,
24             yerr=sigma_T, fmt='o',
25             capsize=3, label='Datos')
26 plt.plot(t_fine, T_fit_fine,
27         'r-', label='Ajuste')
28 plt.xlabel('Tiempo (min)')
29 plt.ylabel('Temperatura (°C)')
30 plt.legend()
31 plt.grid(True, alpha=0.3)
32 plt.show()
```

Discusión: Los parámetros ajustados deben estar dentro de $\pm 2\sigma$ de los valores reales.

Soluciones de Referencia $\ni$ Solución 3:

Resistividad vs Temperatura (Parte 1)



```
1 import numpy as np
2 from scipy.optimize import curve_fit
3
4 # Generar datos
5 T = np.array([0, 20, 40,
6              60, 80, 100])
7 rho0_true = 1.68e-8 #  $\Omega \cdot m$ 
8 alpha_true = 0.00393 #  $^{\circ}C^{-1}$ 
9 T0 = 0.0
10
11 # Modelo lineal
12 rho_true = rho0_true * (
13     1 + alpha_true * (T - T0))
14
15 # Incertidumbres variables
16 sigma_rho = (0.02 + 0.0005 * T) * 1e-8
17
18 np.random.seed(7)
```

```
17 rho_obs = rho_true + np.random.normal(
18     0, 1, size=T.size) * sigma_rho
19
20 # Modelo de ajuste
21 def rho_model(T, rho0, alpha):
22     return rho0 * (1 + alpha * T)
23
24 # Ajuste SIN ponderación
25 popt_sin, _ = curve_fit(
26     rho_model, T, rho_obs)
27 rho0_sin, alpha_sin = popt_sin
28
29 print("SIN ponderación:")
30 print(f"   $\alpha = \{alpha\_sin:.6f\}^{\circ}C^{-1}$ ")
```

Soluciones de Referencia $\ni$ Solución 3:

Resistividad vs Temperatura (Parte 2)



```
1 # Ajuste CON ponderación
2 popt_con, pcov_con = curve_fit(
3     rho_model, T, rho_obs,
4     sigma=sigma_rho,
5     absolute_sigma=True)
6 rho0_con, alpha_con = popt_con
7
8 # Extraer incertidumbres
9 perr_con = np.sqrt(np.diag(pcov_con))
10 sigma_rho0, sigma_alpha = perr_con
11
12 print("\nCON ponderación:")
13 print(f"   $\alpha$  = {alpha_con:.6f}  $\pm$  "
14       f"{sigma_alpha:.6f}  $^{\circ}\text{C}^{-1}$ ")
15 print(f"   $\alpha$  real = {alpha_true:.6f}")
16
17 # Calcular  $\chi^2$  reducido
18 rho_fit = rho_model(T, rho0_con,
19                      alpha_con)
20 res = rho_obs - rho_fit
21 chi2 = np.sum((res/sigma_rho)**2)
22 nu = T.size - 2
23 chi2_red = chi2/nu
24
25 print(f"   $\chi^2_{\text{red}} = \{chi2\_red:.2f\}")
26
27 # Comparación
28 print("\nComparación:")
29 diff_sin = abs(alpha_sin -
30                alpha_true)
31 diff_con = abs(alpha_con -
32                alpha_true)
33 print(f"Error sin pond: {diff_sin:.6f}")
34 print(f"Error con pond: {diff_con:.6f}")$ 
```

Discusión: El ajuste ponderado da resultados más cercanos al valor real cuando las incertidumbres varían.

Soluciones de Referencia $\ni$ Solución 4:

Análisis de Residuos (Parte 1)



```
1 # Usar datos del Ejercicio 1 (péndulo)
2 import matplotlib.pyplot as plt
3
4 # Calcular residuos
5 residuos = T_obs - T_fit
6
7 # Estadísticas básicas
8 media_res = np.mean(residuos)
9 std_res = np.std(residuos, ddof=1) # ddof=1 para muestra
10
11 print("Estadísticas de residuos:")
12 print(f"Media = {media_res:.6f} s (debe ser  $\approx 0$ )")
13 print(f"Desv. estándar = {std_res:.4f} s (esperado: {sigma:.4f} s)")
14 print(f"Residuo máximo = {np.max(np.abs(residuos)):.4f} s")
15
16 # Residuos normalizados
17 res_norm = residuos / sigma
18 print(f"\nResiduos normalizados:")
19 print(f"Media = {np.mean(res_norm):.4f}")
20 print(f"Desv. estándar = {np.std(res_norm, ddof=1):.4f} (esperado:  $\approx 1$ )")
```

Soluciones de Referencia $\ni$ Solución 4:

Análisis de Residuos (Parte 2)



```
1 # Crear gráficos de diagnóstico
2 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
3
4 # Panel 1: Residuos vs L
5 axes[0].scatter(L, residuos, s=50, alpha=0.6)
6 axes[0].axhline(0, color='r', linestyle='--', linewidth=2)
7 axes[0].axhline(sigma, color='gray', linestyle=':', label='±σ')
8 axes[0].axhline(-sigma, color='gray', linestyle=':')
9 axes[0].set_xlabel('Longitud L (m)')
10 axes[0].set_ylabel('Residuos (s)')
11 axes[0].set_title('Residuos vs Variable Independiente')
12 axes[0].legend()
13 axes[0].grid(True, alpha=0.3)
14
15 # Panel 2: Histograma de residuos normalizados
16 axes[1].hist(res_norm, bins=5, alpha=0.7, edgecolor='black', density=True)
17 x_gauss = np.linspace(-3, 3, 100)
18 axes[1].plot(x_gauss, (1/np.sqrt(2*np.pi))*np.exp(-x_gauss**2/2),
19             'r-', linewidth=2, label='N(0,1)')
20 axes[1].set_xlabel('Residuos normalizados')
21 axes[1].set_ylabel('Densidad')
22 axes[1].set_title('Distribución de Residuos')
23 axes[1].legend()
24 axes[1].grid(True, alpha=0.3)
25
26 # Panel 3: Q-Q plot (opcional)
27 from scipy import stats
28 stats.probplot(res_norm, dist="norm", plot=axes[2])
29 axes[2].set_title('Q-Q Plot')
30 axes[2].grid(True, alpha=0.3)
31
```

Consolidación y Retroalimentación

Consolidación y Retroalimentación $\ni$ Puesta en Común - ¿Cómo les fue?

- ¿Qué métrica resultó más difícil de calcular o interpretar?
- ¿Encontraron diferencias significativas entre ajustes ponderados y no ponderados?
- ¿Qué revela el análisis de residuos sobre la calidad del modelo?
- ¿Alguna dificultad con la extracción de incertidumbres desde `pcov`?

Objetivo

Consolidar la comprensión de diagnóstico riguroso de ajustes antes de la evaluación.

Consolidación y Retroalimentación $\ni$ Errores Comunes en Ajuste de Datos

Errores frecuentes

- **Olvidar `absolute_sigma=True`:** Las incertidumbres quedan mal escaladas
- **No verificar residuos:** Patrones sistemáticos indican modelo inadecuado
- **Interpretar mal χ^2_{red} :** Valor muy bajo puede indicar sobreestimación de σ
- **Ignorar la matriz de covarianza:** Perder información sobre correlación entre parámetros
- **No ponderar cuando σ_i varía:** Sesgo hacia puntos con mayor incertidumbre

Consejo

Siempre realizar análisis completo: ajuste $\rightarrow$ métricas $\rightarrow$ residuos $\rightarrow$ interpretación física.

Conclusiones

- **Repasamos** conceptos fundamentales de ajuste de datos:
 - `np.polyfit` vs `curve_fit`
 - Métricas: residuos, R^2 , χ^2_{red}
 - Ponderación por incertidumbres
- **Practicamos** aplicaciones físicas:
 - Péndulo simple
 - Enfriamiento de Newton
 - Resistividad eléctrica
- **Desarrollamos** habilidades de diagnóstico mediante análisis de residuos

Habilidad adquirida

Capacidad de realizar ajustes rigurosos con evaluación crítica de la calidad del modelo y manejo apropiado de incertidumbres.

- **Sesión siguiente (P2):** Ejercicios prácticos integradores con casos más complejos
- **Recomendación de estudio:**
 - Repasar documentación de `scipy.optimize.curve_fit`
 - Practicar análisis de residuos con sus propios datos
 - Revisar interpretación física de χ^2_{red}
- **Preparación para evaluación:** Dominar el flujo completo de ajuste y diagnóstico